

Punto 1: Descripción del problema

Un personaje ingresa a una arena de combate por turnos con vida, energía y una habilidad especial. Debe derrotar enemigos, administrar sus recursos y decidir el momento exacto para intentar salir victorioso antes de que se agoten sus turnos o su vida. El programa valida cada acción del jugador, actualiza sus estadísticas y determina al final si logró la victoria o fue derrotado.

Punto 2: Tabla de variables

Variable	Tipo de dato	Función
Vida	int	Representa los puntos de salud del jugador (inicia en 12, máximo 12).
energia	int	Representa los puntos de energía disponibles para acciones (inicia en 5, máximo 5).
enemigos_derrotados	int	Contador de enemigos eliminados durante la partida.
pociones	int	Cantidad de pociones de vida disponibles (inicia en 1).
turnos	int	Contador de turnos consumidos por el jugador.
habilidad_especial_disponible	bool	Indica si la habilidad especial aún puede usarse (true al inicio).
victoria	bool	Bandera que determina si el jugador ganó la partida (false al inicio).
Max_turnos_permitidos	const int	Límite máximo de turnos permitidos (valor fijo: 10).

Punto 3: Condición del while en lenguaje natural

El ciclo while se ejecuta **mientras se cumplan simultáneamente** estas cuatro condiciones:

1. Los turnos usados sean **menores** al máximo permitido (10).
2. La vida del jugador sea **mayor** que cero.
3. La energía del jugador sea **mayor o igual** a cero.
4. La variable victoria sea **falsa** (es decir, que el jugador aún no haya ganado).

Si **alguna** de estas condiciones deja de cumplirse, el ciclo termina y la partida finaliza.

Punto 4: Uso de los operadores lógicos `&&` y `||`

Operador `&&` (Y lógico)

Se utiliza en **dos lugares principales**:

a) En la condición del `while`

```
while (turnos < Max_turnos_permitidos && Vida > 0 && energia >= 0 && victoria == false)
```

Aquí el `&&` exige que **todas** las condiciones sean verdaderas al mismo tiempo para que el ciclo continúe. Si falla una sola, el juego termina.

b) Dentro de cada condición de victoria (`case 5`)

```
(enemigos_derrotados >= 5 && energia > 0)
```

```
(enemigos_derrotados >= 4 && habilidad_especial_disponible == false)
```

En ambos casos, el `&&` indica que deben cumplirse **ambos** requisitos juntos.

Operador `||` (O lógico)

Se utiliza en la condición de victoria del `case 5`:

```
if ((enemigos_derrotados >= 5 && energia > 0) || (enemigos_derrotados >= 4 && habilidad_especial_disponible == false))
```

El `||` separa dos caminos posibles para ganar: basta con que se cumpla **una** de las dos condiciones (la A o la B) para que el jugador sea victorioso.

Punto 5: Dos pruebas realizadas y su resultado

Prueba A — Victoria por uso de habilidad especial

Paso	Acción	Vida	Energía	Enemigos	Pociones	Habilidad	Turnos	Resultado
Inicio	—	12	5	0	1	Sí	0	Estado inicial
1	Habilidad especial	12	2	2	1	No	1	Derrota 2 enemigos, consume 3 energía
2	Ataque básico	11	1	3	1	No	2	+1 enemigo, -1 energía, -1 vida
3	Ataque básico	10	0	4	1	No	3	+1 enemigo, -1 energía, -1 vida
4	Recuperar energía	9	2	4	1	No	4	+2 energía, -1 vida
5	Intentar salir victorioso	9	2	4	1	No	5	Victoria = true (cumple condición B: enemigos ≥ 4 y habilidad ya usada)

Resultado: El jugador gana en el turno 5. El programa imprime "*¡Has ganado la partida!*" y el ciclo finaliza.

Prueba B — Derrota por agotamiento de vida

Paso	Acción	Vida	Energía	Enemigos	Pociones	Habilidad	Turnos	Resultado
Inicio	—	12	5	0	1	Sí	0	Estado inicial

Paso	Acción	Vida	Energía	Enemigos	Pociones	Habilidad	Turnos	Resultado
1	Recuperar energía	11	5	0	1	Sí	1	Energía ya al máximo, solo recibe daño
2	Recuperar energía	10	5	0	1	Sí	2	Ídem
3	Recuperar energía	9	5	0	1	Sí	3	Ídem
4	Recuperar energía	8	5	0	1	Sí	4	Ídem
5	Recuperar energía	7	5	0	1	Sí	5	Ídem
6	Recuperar energía	6	5	0	1	Sí	6	Ídem
7	Recuperar energía	5	5	0	1	Sí	7	Ídem
8	Recuperar energía	4	5	0	1	Sí	8	Ídem
9	Recuperar energía	3	5	0	1	Sí	9	Ídem
10	Recuperar energía	2	5	0	1	Sí	10	turnos ya no es < 10

Resultado: Al finalizar el turno 10, la condición `turnos < Max_turnos_permitidos` es falsa, por lo que el `while` termina. Como `victoria` sigue siendo `false`, el programa evalúa el `if` final y muestra el mensaje de derrota.